

Shimera - Track Technique EIP

Choix des objectifs

Le projet: Shimera est une bibliothèque en C++23 qui permet aux développeurs d'ajouter des effets visuels GPU par-dessus différents types de projets graphiques, sur différentes bibliothèques graphiques sans toucher à la programmation GPU.

Objectifs retenus par le groupe: les objectifs obligatoires 1 et 2, et les objectifs complémentaires A et D.

Objectif 1: Évaluer et intégrer de nouvelles technologies (veille technologique)

L'objectif

L'objectif est de mener une veille technologique sur l'ensemble des technologies utilisées par Shimera (C++, OpenGL, GLSL, xmake,). De réaliser des analyses et des POC de ces technologies pour prendre les meilleures décisions. Partager ces analyses fait également partie de l'objectif.

Pertinence pour la track technique

Le but est d'apprendre à suivre les évolutions technologiques et à choisir les outils les plus adaptés en les justifiant, plutôt que d'utiliser par habitude ce que l'on connaît déjà. Pour y arriver, il faut suivre une veille rigoureuse, cela oblige à comparer chaque solution avec d'autres options crédibles et à justifier ses choix sur la durée.

Pertinence pour Shimera

Shimera repose sur une stack graphique complexe, fortement dépendante des pilotes et de bibliothèques dont les évolutions peuvent entraîner des ruptures de compatibilité. Une veille technologique est donc essentielle afin d'anticiper ces évolutions et de protéger les utilisateurs avant qu'elles n'impactent leurs projets.

Preuves concrètes

Nous avons mis en place une veille technologique documentée dans le dossier "docs/private/docs/technology_watch" ou dans "docs/private/docs/fr/technology_watch" pour la version française. Cette veille technologique nous a permis de corriger certains problèmes, mais aussi de prévoir le futur de la bibliothèque. Par exemple, la potentielle deuxième version de la bibliothèque se base sur des informations trouvées sur Reddit. De

même pour notre choix de vouloir passer les shaders sur le langage Slang.

Veille technologique Shimera

Comment l'équipe reste informée sur les technologies utilisées par Shimera, et transforme cette information en décisions concrètes.

Objectif

Shimera est une librairie C++ bas niveau qui superpose des effets de post-processing (shaders) au-dessus d'OpenGL, SFML et raylib. Ce sont des technologies matures et stables, mais la pile est vaste : les APIs graphiques sont larges et truffées de comportements spécifiques aux pilotes, et l'outillage autour (librairies de fenêtrage, système de build, CI/CD, documentation) publie encore quelques ruptures ponctuelles.

La veille technologique est notre effort organisé pour maîtriser cette surface. L'objectif n'est pas de réagir à un flux de changements permanent, mais de garder un petit ensemble de canaux à fort signal sous revue régulière, afin de :

- repérer les ruptures d'API et les dépréciations avant qu'elles n'atteignent nos utilisateurs ;
- adopter les bonnes pratiques de la communauté plutôt que de les réinventer ;
- garder une structure de dépôt et un déploiement alignés sur la façon dont les librairies graphiques open source comparables sont construites ;
- garder justifiés dans le temps nos choix techniques (documentés dans les [Comparatifs technologiques](#)).

Ce document décrit les quatre canaux que nous surveillons et la manière dont chacun alimente le projet.

Périmètre : technologies surveillées

La veille est centrée sur les technologies réellement utilisées dans le dépôt.

Domaine	Technologie	Pourquoi nous la surveillons
Langage	C++23	Nouvelles fonctionnalités du standard, support des compilateurs (GCC/Clang/MSVC), patterns RAI pour les ressources GPU
API graphique	OpenGL 3.3+, GLEW	Support des extensions, comportement des pilotes, dépréciation des chemins legacy
Shaders	GLSL	Compatibilité des versions, pièges de portabilité entre pilotes
Mathématiques	GLM	Changements d'API, considérations d'alignement/SIMD
Backends	SFML 3, raylib, SDL3 (exploratoire)	Versions majeures et ruptures d'API
Build	xmake	Mises à jour du gestionnaire de paquets, configuration des toolchains, builds multi-plateformes
CI/CD & hébergement	GitHub Actions, GitHub Releases	Montées de version des actions, changements d'images de runners, patterns de release
Documentation	VitePress	Mises à jour du canal alpha, changements du format de configuration

Aperçu de la documentation sur la veille technologique

De plus, grâce à celle-ci nous avons aussi fait une note comparative au début du projet qui nous a permis d'avoir des technologies cohérentes pour la suite. Celle-ci est documentée dans le fichier “docs/private/docs/library/technology_comparative_briefs.md” pour la version anglaise et dans le fichier “docs/private/docs/fr/library/technology_comparative_briefs.md”.

Documentation et comparatifs des technologies utilisées dans la librairie

Objectif

Ce document résume les technologies principales de Shimera et fournit des comparatifs courts pour justifier les choix d'architecture.

Shimera est une librairie C++ conçue pour permettre à ses utilisateurs (principalement techniciens) d'implémenter des effets visuels au-dessus de leurs projets graphiques sans nécessité de connaissances GPU. La librairie fournit un ensemble de shaders facilement intégrables dans des projets OpenGL, SFML et Raylib. Chaque shader est accompagné d'une fonction appellable qui permet aux utilisateurs d'implémenter et de modifier les paramètres du shader selon leurs besoins.

Vue d'ensemble technologique

Langage et exécution

- C++23: langage principal pour les performances, la gestion RAII et l'interopérabilité bas niveau.
- GLSL: langage shader utilisé pour les effets sur GPU.

Build et outillage

- xmake: outil de build principal dans ce dépôt.

Comparatif 1: Langage principal de la librairie

C++ vs C vs Rust vs C# vs Java vs Python

Critère	C++	C	Rust	C#	Java	Python
Performance bas niveau	Excellente	Excellente	Excellente	Bonne à très bonne	Bonne (JIT)	Faible à moyenne
Interop directe avec OpenGL/SFML/raylib/SDL	Excellente	Bonne (SDL très naturel, SFML moins naturel)	Bonne (FFI nécessaire selon libs)	Moyenne (bindings/couches interop)	Moyenne (LWJGL/JOGL pour OpenGL, bindings tiers pour raylib/SFML)	Moyenne (bindings, couches C/C++ fréquentes)
Contrôle mémoire et ressources GPU	Excellente (RAII mature)	Bonne mais plus manuelle	Excellente (ownership/borrow checker)	Moyenne (GC, patterns natifs nécessaires)	Moyenne (GC, gestion indirecte des ressources)	Faible à moyenne (gestion indirecte via)

Aperçu de la documentation du comparatif des technologies utilisées dans la bibliothèque

Enfin, lorsque des décisions techniques devaient être prises, nous avons développé des POC (preuves de concept) pour orienter nos choix. Ceux-ci sont documentés dans le dossier “docs/private/docs/poc” pour leur version anglaise et dans le dossier “docs/private/docs/fr/poc” pour leur version française.

Injection OpenGL 5 Mars 2025

Vue d'ensemble

Ce document est le résultat d'un POC (Proof of Concept) visant à démontrer comment injecter des shaders OpenGL dans une application utilisant SFML/SDL/Raylib tout en encapsulant tous les appels OpenGL dans une bibliothèque dédiée. L'objectif est de prouver que nous pouvons créer une bibliothèque générique utilisable avec plusieurs bibliothèques graphiques sans exposer les détails d'OpenGL à l'utilisateur final.

Stack technique

- Langage: C++
- Bibliothèques graphiques: SFML, SDL, Raylib
- Version OpenGL: 3.3 Core Profile
- Système de build: CMake

Approche technique

Nous avons commencé par créer des applications exemples utilisant SFML, SDL et Raylib. Chaque application initialise son contexte graphique respectif, sa fenêtre et dessine des formes basiques en utilisant leurs méthodes de rendu natives.

Ensuite, nous avons développé une bibliothèque OpenGL dédiée qui encapsule tous les appels OpenGL. Cette bibliothèque fournit une API simple pour initialiser OpenGL, compiler des shaders et rendre des objets. La bibliothèque est conçue pour être indépendante de la bibliothèque graphique utilisée par l'application.

Après cela, nous avons tenté de récupérer le contexte de chaque bibliothèque graphique et de le transmettre à notre bibliothèque OpenGL. Puis, nous avons essayé d'appliquer des shaders de post-traitement au contexte créé par chaque bibliothèque graphique et de rendre la sortie finale à l'écran en utilisant OpenGL pour démontrer que notre bibliothèque peut fonctionner de manière générique avec différentes bibliothèques graphiques.

Résultats

Le POC n'a pas réussi à atteindre le résultat souhaité. Bien que nous ayons pu créer la bibliothèque OpenGL dédiée et compiler les shaders, nous avons rencontré des défis importants lors de l'intégration avec les bibliothèques graphiques.

1. **Gestion du contexte:** Certaines bibliothèques graphiques comme Raylib gèrent leur propre contexte OpenGL, ce qui rend difficile le partage du contexte avec notre bibliothèque OpenGL. Cela a entraîné des problèmes où les appels OpenGL effectués par notre bibliothèque n'affectaient pas le rendu effectué par les bibliothèques graphiques.
2. **Compatibilité des versions:** C'est un exemple spécifique, SDL n'utilise pas nécessairement OpenGL pour le rendu. Il choisit le meilleur backend disponible sur le système (Direct3D, Vulkan, Metal, etc.) et si nous le forçons à utiliser OpenGL, nous ne pouvons utiliser SDL que pour créer une fenêtre, créer un contexte et gérer les événements d'entrée. Cela signifie que tout le rendu doit être effectué en utilisant des appels OpenGL, ce qui va à l'encontre de l'objectif d'utiliser les capacités de rendu de SDL.

Aperçu de la documentation du POC sur l'injection OpenGL

20/01/2026 - Couche d'Abstraction : Première Tentative (Basée sur du CRTP)

Vue d'Ensemble

Avant d'arriver à la couche d'abstraction actuelle basée sur la factory, nous avons implémenté une approche **CRTP (Curiously Recurring Template Pattern)**. Cette première tentative était **single-pass uniquement** et se concentrait sur la fonctionnalités d'une API simple et de haut niveau pour appliquer un effet de post-traitement.

Statut : Conservé dans le code source à des fins historiques/de preuve de travail uniquement (pour l'instant, devrait être supprimé dans le futur).

Fichiers :

- `PostProcessingPipelineBase.inl` - Classe de base CRTP
- `PostProcessingPipeline.inl` - Implémentation SFML

Architecture

PostProcessingPipelineBase (Base CRTP)

Objectif : Fonctionnalité partagée entre toutes les implémentations de backend utilisant le patron CRTP pour le polymorphisme à la compilation.

Code :

```
template<typename C>
class PostProcessingPipelineBase {
public:
    C& setUniform(const std::string& name, const UniformValue& value) {
        uniforms[name] = value;
        static_cast<C*>(this)->applyUniform(name, value);
        return *static_cast<C*>(this); // Enables method chaining
    }

    void resize(int w, int h) {
        width = w;
        height = h;
        static_cast<C*>(this)->onResize(w, h);
    }

protected:
    std::map<std::string, UniformValue> uniforms;
    int width = 0;
    int height = 0;
};
```

Aperçu de la documentation du POC sur la tentative de couche d'abstraction CRTP

Raylib avec les shaders Shimera - 22 Janvier 2026

Vue d'ensemble

Ce document décrit un Proof of Concept (POC) visant à intégrer des shaders personnalisés dans une application utilisant la bibliothèque Raylib. L'objectif est de démontrer comment appliquer des effets graphiques avancés en utilisant des shaders GLSL tout en exploitant les fonctionnalités de Raylib pour la gestion de la fenêtre et du contexte graphique.

Stack technique

- **Langage:** C++
- **Bibliothèque graphique:** Raylib
- **Version OpenGL:** 3.3 Core Profile
- **Système de build:** XMake

Approche technique

Nous avons commencé par créer une application de base utilisant Raylib pour initialiser une fenêtre et dessiner des formes simples. Ensuite, nous avons utilisé les fonctionnalités d'OpenGL et les shaders post-traitement que nous avons développés dans le cadre de la librairie Shimera pour appliquer des effets visuels sur le rendu final de l'application.

L'application suit les étapes suivantes :

1. Initialisation de Raylib et création d'une fenêtre.
2. Chargement et compilation des shaders GLSL pour les effets de post-traitement.
3. Rendu de la scène de base avec Raylib.
4. Application des shaders de post-traitement sur le rendu de la scène.
5. Affichage du résultat final à l'écran.

Résultats

Le POC a réussi à intégrer les shaders personnalisés dans l'application Raylib. Nous avons pu appliquer des effets de post-traitement tels que la distorsion en utilisant les fonctionnalités de la Raylib pour afficher des formes simples et gérer le contexte graphique. Pour la compilation et l'exécution des shaders, nous avons utilisé les fonctions OpenGL directement, que nous avons intégrées dans une texture de rendu créée par Raylib.

Conclusion

Le POC a démontré avec succès la possibilité d'intégrer des shaders personnalisés dans une application Raylib. En utilisant les capacités de Raylib pour la gestion de la fenêtre et du contexte graphique, ainsi que les fonctionnalités d'OpenGL pour le rendu des shaders, nous avons pu créer des effets visuels avancés.

Ce travail ouvre la voie à une utilisation de la librairie Shimera avec Raylib, permettant aux développeurs de tirer parti des shaders personnalisés

Aperçu de la documentation du POC sur l'intégration de nos shaders dans une application utilisant la bibliothèque Raylib

Objectif 2: Structurer, documenter et durcir l'architecture technique

L'objectif

L'objectif est de présenter et de justifier l'architecture de Shimera (composants, interfaces, flux de données, build, déploiement), de fournir une documentation complète (README structuré, plus des documents avancés et des diagrammes), et de durcir le code par des standards imposés, des outils de qualité, des tests sur les modules critiques, ainsi que de la gestion d'erreurs et du logging.

Pertinence pour la track technique

La track accorde une grande importance à la solidité, à la clarté et à la durabilité du code, bien au-delà du simple fait que le programme “fonctionne”. Cet objectif permet de mettre ces principes en pratique. Il sert aussi à éviter les erreurs classiques que la track cherche à prévenir : l'absence de documentation (README), une structure rigide et difficile à modifier, ou des choix technologiques faits au hasard.

Pertinence pour Shimera

Pour une bibliothèque comme Shimera, l'architecture est le cœur du produit. Cet objectif nous aide à structurer un projet technique, à expliquer clairement son fonctionnement, et à maintenir une bonne qualité de code tout au long du développement.

Preuves concrètes

Pour ce faire nous avons documenté l'architecture de Shimera, document que vous pouvez retrouver avec ce lien: [Shimera Architecture Diagram | ShimeraDocs](#)

ShimeraDocs

UserDoc

DevDoc

%

🔍

🌙

👤

DevDoc

Git Workflow

Architecture

Architecture Diagram

Code Standards

Shimera Architecture Diagram

A visual overview of Shimera.

Contents:

- [1. Interaction layers & core modules](#)
- [2. Backend abstraction \(interfaces → implementations\)](#)
- [3. Rendering flow](#)
- [4. External dependencies](#)
- [5. Module reference table](#)

1. Interaction layers & core modules

Dependency direction is strictly one-way (top → bottom): high-level effect orchestration never depends on a concrete backend. A backend is chosen at compile time, not at runtime.

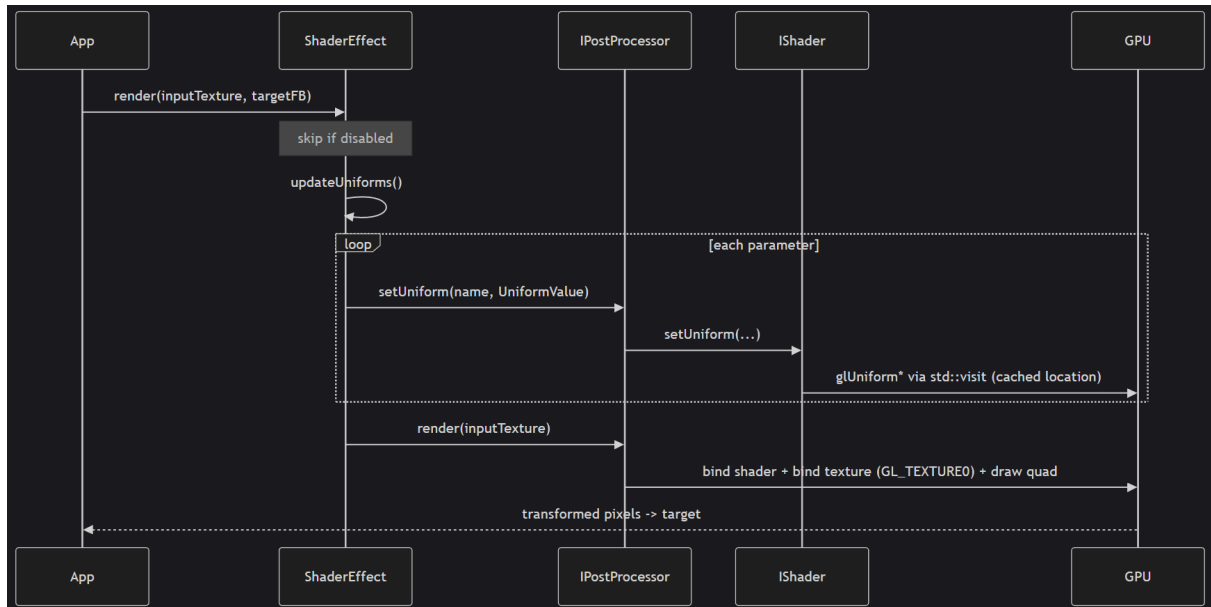
```

graph TD
    subgraph Application_Layer [Application Layer: host apps / examples]
        direction TB
        A1["OpenGL example  
+ GLFW window / context"]
        A2["SFML example"]
        A3["Raylib example"]
    end
    subgraph Public_API [Public API / Facade]
        direction TB
        P1["shimera.h  
umbrella include"]
        P2["shimera_epl.h  
SHIMERA_API  
export/import macros"]
    end
    subgraph Effect_DSL_Layer [Effect DSL Layer]
        direction TB
        E1["ShadedEffectDesc"]
    end
    Application_Layer --> Public_API
    Public_API --> Effect_DSL_Layer
    
```

On this page

1. Interaction layers & core m...
2. Backend abstraction (inter...
3. Rendering flow
4. External dependencies
5. Module reference table

Documentation complète de l'architecture de Shimera avec plusieurs diagrammes.



Flux de contrôle et gestion des uniformes par image

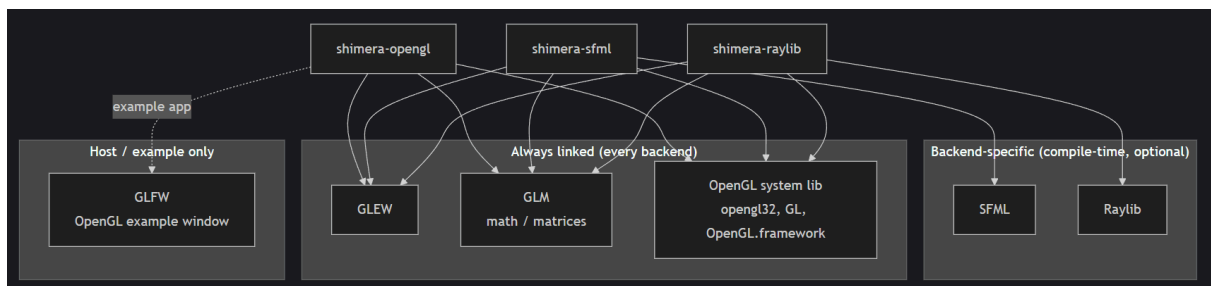


Diagramme représentant les liens avec les dépendances externes



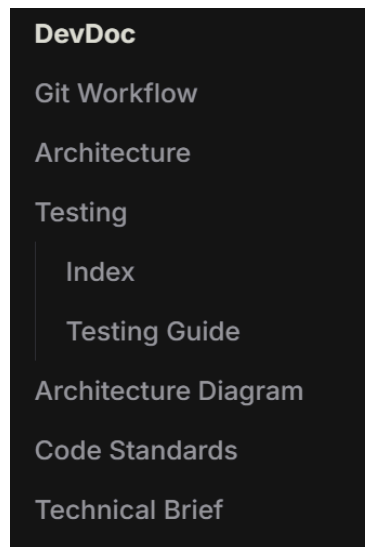
Diagramme des couches d'interaction et des modules principaux

Nous avons aussi mis en place une documentation utilisateur contenant tout ce dont un développeur peut avoir besoin pour utiliser nos shaders. Cette documentation est accessible à ce lien: [UserDoc | ShimerDocs](#). À noter que la langue peut être changée en français ou en anglais en haut à droite de la page.

ShimerDocs	
UserDoc	
Installation	
Why Shimer	
Performance	
Effect Pipeline Builder	
Materials	
Index	
Fresnel	
Shaders	
Index	
Atmospheric Scattering	
Brightness	
Chromatic Aberration	
Colorshift	
Contrast	
Distortion	
Gaussian Blur	
Grayscale	
HDR Bloom	
Saturation	
Vignette	
Pixelisation	

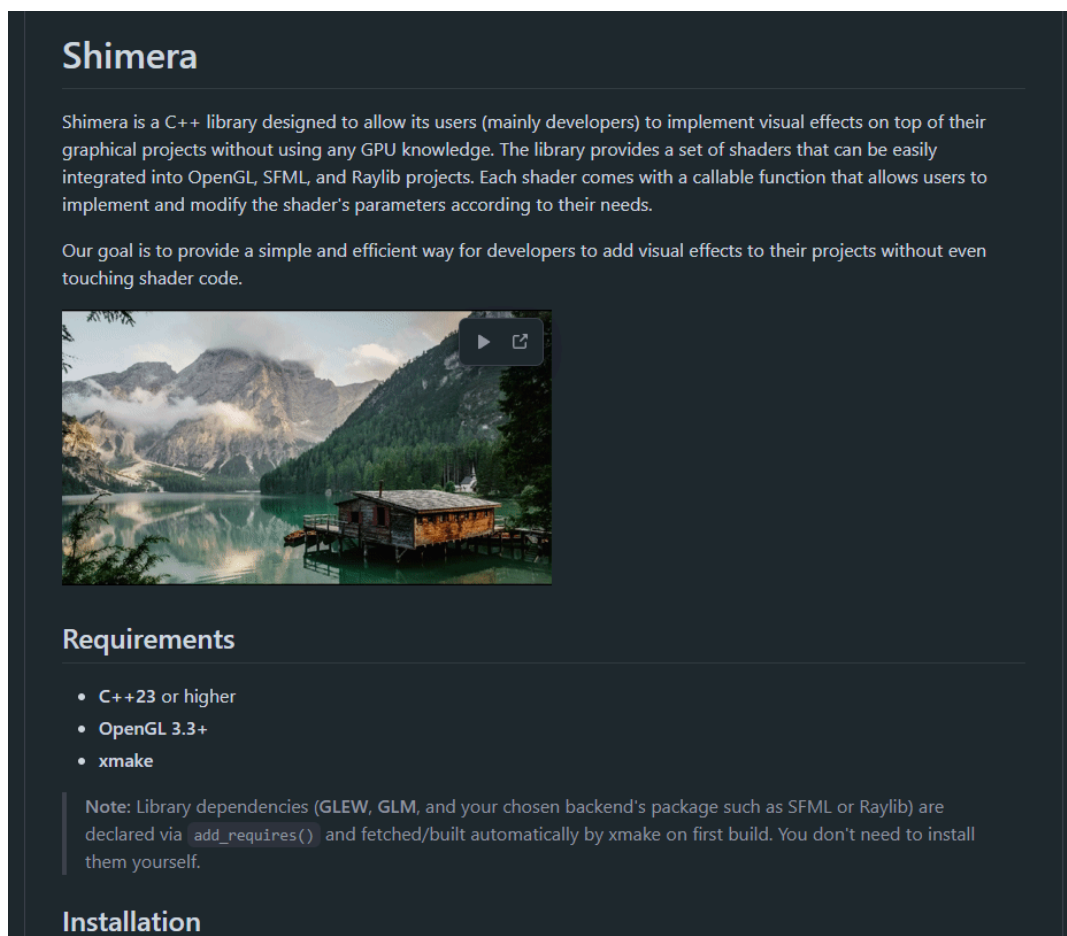
Liste de documents dans la documentation utilisateur.

Pour les personnes voulant contribuer, il existe aussi une documentation plus précise contenant justement l'architecture et d'autres documents pouvant être utiles. Tout cela étant sur la même page web mais dans la section "DevDoc": [DevDoc | Shimeradocs](#)



Liste de documents dans la documentation développeur.

Nous avons aussi un README qui donne les principales informations à savoir, tel qu'une description rapide de la bibliothèque, les prérequis, l'installation, la structure du projet, un lien vers le "CONTRIBUTING.md" et une mention de la licence.



Le début du README du projet dans le dépôt GitHub de Shimeradocs.

De plus nous avons mis en place des Git hooks qui nous permettent de vérifier la norme des commits avec “pre-commit” et de vérifier la propreté de notre code avec “clang-tidy”.

```
> git add src/backend/raylib/RaylibFramebuffer.cpp
> git commit -m "bad commit msg"
Generate compile_commands.json.....Passed
clang-tidy.....Passed
Validate commit format.....Failed
- hook id: validate-commit-msg
- exit code: 1

Invalid commit format!

Expected format: <type>(<scope>): <description>

Allowed types:

feat:      Adding new functionality
fix:       Squashing bugs
refactor:  Improving code without changing behavior
docs:      Documentation updates
style:     Formatting and code style fixes
test:      Adding or updating tests
chore:     Maintenance tasks like dependency updates
perf:      Performance optimizations
ci:        Continuous integration tweaks

Examples:

feat(auth): add user login endpoint
fix(dashboard): correct chart rendering issue
docs(readme): update project setup instructions
refactor(utils): extract date formatter function
chore: update dependencies to latest versions

Your message: "bad commit msg"
```

Exemple de commit qui échoue

```
default_install_hook_types: [pre-commit, commit-msg]
repos:
- repo: local
  hooks:
    - id: validate-commit-msg
      name: Validate commit format
      entry: python .github/hooks/validate-commit-msg.py
      language: python
      stages: [commit-msg]

    - id: compile-commands
      name: Generate compile_commands.json
      entry: xmake project -k compile_commands --outputdir=build
      language: system
      pass_filenames: false
      always_run: true
      stages: [pre-commit]

    - id: clang-tidy
      name: clang-tidy
      entry: clang-tidy
      language: system
      args: [ "-p=build" ]
      types: [ c++ ]
      stages: [pre-commit]
```

Configuration de pre-commit

```

report.yaml
1 ---
2 MainSourceFile: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera\include\scene\TransformFactory.hpp'
3 Diagnostics:
4   - DiagnosticName: clang-diagnostic-error
5     DiagnosticMessage:
6       Message: 'use of class template ''Vec3'' requires template arguments'
7       FilePath: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera\include\scene\TransformFactory.hpp'
8       FileOffset: 1130
9       Replacements: []
10    Notes:
11      - Message: template is declared here
12        FilePath: 'include\uniform\Vec3.inl'
13        FileOffset: 895
14        Replacements: []
15        Ranges:
16          - FilePath: 'include\uniform\Vec3.inl'
17            FileOffset: 866
18            Length: 21
19    Level: Error
20    BuildDirectory: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera'
21   - DiagnosticName: clang-diagnostic-error
22     DiagnosticMessage:
23       Message: 'use of class template ''Vec3'' requires template arguments'
24       FilePath: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera\include\scene\TransformFactory.hpp'
25       FileOffset: 1194
26       Replacements: []
27    Notes:
28      - Message: template is declared here
29        FilePath: 'include\uniform\Vec3.inl'
30        FileOffset: 895
31        Replacements: []
32        Ranges:
33          - FilePath: 'include\uniform\Vec3.inl'
34            FileOffset: 866
35            Length: 21
36    Level: Error
37    BuildDirectory: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera'
38   - DiagnosticName: clang-diagnostic-error
39     DiagnosticMessage:
40       Message: 'no template named ''variant'' in namespace ''std''
41       FilePath: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera\include\uniform\Uniform.inl'
42       FileOffset: 1091
43       Replacements: []
44    Ranges:
45      - FilePath: 'C:\Users\eddyg\Documents\Epitech\EIP\Shimera\include\uniform\Uniform.inl'
46        FileOffset: 1086
47        Length: 5

```

Exemple de rapport de clang-tidy

Nous avons aussi une documentation sur les standards de code et une autre sur le flux de travaux Git: [Shimera - Coding Standards | ShimeraDocs](#), [Git Workflow Guide | ShimeraDocs](#)

Shimera - Coding Standards

This document defines the coding conventions used across the Shimera project. Its goal is to ensure consistency, readability, and maintainability across the entire codebase.

1. Naming Conventions

General Rules

- Use descriptive names (avoid abbreviations unless widely understood)
- English for everything (variables, functions, comments)

Naming Styles

Element	Style	Example
Classes / Structs	<code>PascalCase</code>	<code>Framebuffer</code>
Functions / Methods	<code>camelCase</code>	<code>bind()</code>
Variables	<code>camelCase</code>	<code>width</code>
Constants	<code>UPPER_SNAKE_CASE</code>	<code>FRAMES</code>

On this page

- 1. Naming Conventions
- 2. File Organization
- 3. Class Structure
- 4. Modern C++ Conventions
- 5. Documentation
- 6. General Guidelines
- 7. Automatic Checks with cla...
- References

Outil Capture d'écran

Capture d'écran copiée dans le Presse-papiers
Enregistrement automatique dans le dossier des captures d'écran.

Balisage et partage

Aperçu de la documentation sur les standards de code

Git Workflow Guide

Introduction

This guide centralizes the contribution rules for developers working on the project. It combines branch naming, commit conventions, and Git hooks setup in a single place.

1. Branch Naming

We use a conventional branching strategy to keep the repository clean and easy to navigate.

Use one of these prefixes:

- `main/` : stable production branch
- `dev/` : integration and ongoing development
- `feat/` : new feature or enhancement
- `fix/` : bug fix
- `hotfix/` : urgent production fix
- `chore/` : maintenance tasks (dependencies, docs, tooling)

On this page

- Introduction
- 1. Branch Naming
- 2. Commit Messages
- 3. Git Hooks With pre-commit
- 4. Recommended Contributio...

Aperçu de la documentation sur les flux de travaux

Pour finir, la bibliothèque est déployée et versionnée automatiquement grâce à des GitHub Actions. Le déploiement se fait sur Windows et Linux.

Shimera 0.3.6 Latest

 github-actions released this 7 hours ago  v0.3.6  f4d6d81

Changelog

This project is in **beta**. The first official version tag (**0.3.6**) marks the start of the beta phase.

This changelog follows [Keep a Changelog](#) conventions with [Conventional Commits](#).

Licensing & Open Source (Jun - Jul 2026)

Added

- GNU General Public License v3.0 added to the repository ([029bfd1](#))
- SPDX license headers added to all source files and shaders ([03a70aa](#))
- GPL-3.0 license header added to remaining files ([90c5f09](#))
- Licensing information and updated contribution guidelines ([55d008e](#) / [87d1d26](#))

Materials, Fresnel & Atmospheric Scattering (Jun 2026)

Added

- Full OpenGL Material support, partial RayLib Material support, and Fresnel Effect Material shader ([cae5ddc](#))
- Atmospheric Scattering effect with depth buffer support ([2eda0d6](#))
- Effect pipeline compatibility with depth texture ([3a1b031](#))

Fixed

- `RayLibConvert` empty class removed ([2f74d64](#))
- Duplicate includes and member variable removed from `SFMLPostProcessor` ([39a5131](#))
- Wrong `#ifndef` guards renamed across older files ([f1441a3](#))

HDR Bloom, Gaussian Blur & Pixelation (May - Jun 2026)

Exemple de changelog

▼ Assets 11

-  [shimera-0.3.6-opengl-linux-x64.tar](#)
-  [shimera-0.3.6-opengl-windows-x64.zip](#)
-  [shimera-0.3.6-opengl-windows-x86.zip](#)
-  [shimera-0.3.6-raylib-linux-x64.tar](#)
-  [shimera-0.3.6-raylib-windows-x64.zip](#)
-  [shimera-0.3.6-raylib-windows-x86.zip](#)
-  [shimera-0.3.6-sfml-linux-x64.tar](#)
-  [shimera-0.3.6-sfml-windows-x64.zip](#)
-  [shimera-0.3.6-sfml-windows-x86.zip](#)
-  [Source code \(zip\)](#)
-  [Source code \(tar.gz\)](#)

Déploiement de la bibliothèque sur Windows et Linux

Objectif A: Protéger et valoriser sa technologie

L'objectif

L'objectif est d'identifier les éléments du projet à protéger, déposer une preuve d'antériorité datée, mettre en place une protection technique et légale, documenter la stratégie de protection, et valoriser à l'extérieur la valeur technique unique du projet sans exposer ce qui doit rester protégé.

Pertinence pour la track technique

Une track technique ne consiste pas seulement à créer des systèmes ou à écrire du code. Il est également important de savoir le protéger que ce soit d'un point de vue légal ou technique comme on devrait le faire pour n'importe quel autre produit.

Pertinence pour Shimera

Shimera est entièrement open source, ce qui fait de la notion de "protection" une stratégie réfléchie plutôt qu'une contradiction. L'objectif n'est pas d'empêcher l'utilisation ou la modification du code, mais de définir clairement les droits accordés aux utilisateurs tout en préservant ceux des auteurs. Le but est de garantir que le projet peut évoluer de manière collaborative tout en assurant la reconnaissance du travail réalisé et en facilitant la résolution d'éventuels litiges sur l'antériorité des développements.

Preuves concrètes

Pour commencer, notre stratégie de protection juridique est documentée. Vous pouvez la trouver dans le dossier "docs/private/docs/legal" pour la version anglaise ou dans le dossier "docs/private/docs/fr/legal" pour la version française.

Stratégie de protection juridique de Shimera

Ce document décrit comment Shimera protège ses auteurs, son code et ses options de licence futures. Il complète la [Stratégie de diffusion](#), qui couvre le modèle de distribution open source, et s'appuie sur les mêmes fondations juridiques déclarées dans le dépôt Shimera : `LICENSE` (`GPL-3.0-only`), `AUTHORS.md` et `CONTRIBUTING.md`.

Shimera est développé et distribué ouvertement. Être open source ne signifie **pas** renoncer à la protection : cela signifie choisir quels droits conserver, lesquels concéder, et comment prouver la paternité. Cette stratégie rend ces choix explicites.

1. Ce que nous protégeons, et contre quoi

Actif	Menace	Instrument principal
Paternité (qui a écrit le code)	Un tiers revendiquant le code ou l'idée comme siens	Droit d'auteur automatique + antériorité prouvable (e-Soleau, historique Git, Software Heritage)
Antériorité (nous avons créé en premier, à une date connue)	Un tiers prétendant à une création antérieure, ou contestant la nôtre	Preuves datées et scellées : dépôt e-Soleau, historique Git public horodaté, archivage indépendant
Intégrité de la licence (le copyleft est respecté)	Un tiers distribuant un dérivé fermé de Shimera	Copyleft GPL-3.0-only + mise en application (résiliation, section 8)
Capacité de relicencier (futur dual-license / MPL)	Perdre le contrôle du droit d'auteur des contributeurs, bloquant tout changement de licence	Cession de droits par les contributions consolidant le relicencier chez les mainteneurs
Le nom « Shimera »	Fork hostile se présentant comme le projet officiel, confusion de marque	Usage public constant, première utilisation documentée, dépôt de marque optionnel

2. Modèle de protection en couches

Aucun instrument unique ne protège un projet logiciel. Shimera empile cinq couches, chacune couvrant ce que les autres ne couvrent pas :

1. **Le droit d'auteur** naît automatiquement à la création, sans formalité.
2. **La licence GPL-3.0-only** transforme ce droit en bouclier actif via le copyleft.
3. **La gouvernance des contributions** consolide le droit de relicencier chez les mainteneurs.
4. **Les preuves d'antériorité** (e-Soleau, historique Git, archivage indépendant) permettent de *prouver* la paternité et la date en cas de contestation.
5. **La protection de la marque** couvre ce que la licence ne couvre pas : le nom du projet.

La suite du document détaille chaque couche.

Début de la documentation sur la stratégie de protection juridique

Nous avons mis notre projet sous la licence GNU GPL-3.0, ce qui pourra évoluer dans le futur.

```
GNU GENERAL PUBLIC LICENSE
Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for
software and other kinds of works.

The licenses for most software and other practical works are designed
to take away your freedom to share and change the works. By contrast,
the GNU General Public License is intended to guarantee your freedom to
share and change all versions of a program--to make sure it remains free
software for all its users. We, the Free Software Foundation, use the
GNU General Public License for most of our software; it applies also to
any other work released this way by its authors. You can apply it to
your programs, too.

When we speak of free software, we are referring to freedom, not
price. Our General Public Licenses are designed to make sure that you
have the freedom to distribute copies of free software (and charge for
them if you wish), that you receive source code or can get it if you
want it, that you can change the software or use pieces of it in new
free programs, and that you know you can do these things.
```

Début de la licence GNU GPL-3.0

```
1 // SPDX-License-Identifier: GPL-3.0-only
2 //
3 // Shimera: a simple way to add visual effects without using any GPU knowledge
4 // Copyright (C) 2025-2026 The Shimera Authors
5 //
6 // This program is free software: you can redistribute it and/or modify
7 // it under the terms of the GNU General Public License as published by
8 // the Free Software Foundation, version 3 of the License.
9 //
10 // This program is distributed in the hope that it will be useful,
11 // but WITHOUT ANY WARRANTY; without even the implied warranty of
12 // MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
13 // GNU General Public License for more details.
14 //
15 // You should have received a copy of the GNU General Public License
16 // along with this program. If not, see <https://www.gnu.org/licenses/>.
17
18 #ifndef SHIMERA_IBACKEND_HPP
19 #define SHIMERA_IBACKEND_HPP
```

Header apposé au début de tous les fichiers source

Par ailleurs, le fichier “CONTRIBUTING.md” rappelle les conditions de la licence et précise qu'en soumettant une contribution, les contributeurs autorisent le projet à modifier ultérieurement la licence applicable à la partie de code qu'ils ont apportée.

Licensing of Contributions

Shimera is released under the **GNU General Public License v3.0** (`GPL-3.0-only`); see the [LICENSE](#) file.

By submitting a contribution (a pull request, patch, or any other form of change), you certify that:

- you wrote the contribution yourself, or otherwise have the right to submit it under the project's license;
- your contribution is provided under the terms of the GPL-3.0; and
- you grant the Shimera maintainers (the people listed in the **Maintainers** section of [AUTHORS.md](#)) a perpetual, worldwide, non-exclusive, royalty-free, irrevocable license to use, reproduce, modify, sublicense and distribute your contribution, **including the right to relicense it, together with the rest of the project, under any other license in the future, whether open source or proprietary/commercial** (for example to dual-license the project as GPL-3.0 plus a commercial license, or to move it to a more permissive open-source license such as MPL-2.0).

This grant lets the project adapt its license over time and pursue dual-licensing while keeping every contribution usable. The license you grant is **non-exclusive**: you keep full copyright to your own contributions and may use them elsewhere as you wish.

New source files should start with the standard license header used across the codebase (`SPDX-License-Identifier: GPL-3.0-only` and the `The Shimera Authors` copyright line).

Passage du “CONTRIBUTION.md”

Authors

Shimera is developed by **The Shimera Authors**: the maintainers and contributors listed below. The `Copyright (C) 2025-2026 The Shimera Authors` notice found in each source file refers collectively to these people. **Each author keeps the copyright to their own contributions.**

Two roles are distinguished:

- **Copyright** is held by every author (maintainers and contributors), each over the part they wrote.
- The **right to relicense the project** is granted to the **maintainers** only. Under the “Licensing of Contributions” section of [CONTRIBUTING.md](#), every contributor grants the maintainers a broad license over their contribution, including the right to change the project's license in the future. The maintainers are therefore the people who can decide a future license change.

Maintainers

The people who steward the project and hold the project-wide license grant described above.

- Léo Maurel ([@Imaoleo](#))
- Paul Arbez ([@PaulArbez](#))
- Eddy Gardes ([@Eddy02136](#))

Contributors

Everyone who has contributed to Shimera. Each keeps the copyright to their own contributions. New contributors are added here as their first contribution is merged.

“Authors.md”

De plus, nous avons déposé un E-Soleau auprès de L'INPI (Institut National de la Propriété Industrielle). C'est un moyen de sceller et de dater notre idée.



RÉPUBLIQUE
FRANÇAISE

Liberté
Égalité
Fraternité



Soleau
RECAPITULATIF DE DEPOT D'UNE
ENVELOPPE ESOLEAU

Numéro national : [REDACTED]

Lieu de dépôt : 92 INPI - Démarche en ligne

Date de la demande : 19/06/2026

Date et heure du dépôt sur le Temps Universel Coordonné (UTC±00:00) : 2026-06-19
14:15:05

Date de fin de validité : 19/06/2031

Référence client :

Intervenants

Titulaires

Nom/Prénom : Monsieur ARBEZ PAUL

Adresse :
[REDACTED]

France

Nom/Prénom : Monsieur GARDES EDDY

Adresse :
[REDACTED]

France

Nom/Prénom : Monsieur MAUREL LÉO

Adresse :
[REDACTED]

France

Récapitulatif du dépôt de notre E-Soleau

Pour ce qui est de l'aspect protection technique, tout est documenté dans le fichier “docs/private/docs/quality/security_map.md” pour la version anglaise et dans le fichier “docs/private/docs/fr/quality/security_map.md” pour la version française.

Shimera - Security Map

This document describes Shimera's technical protection strategy (C++ graphics abstraction library, OpenGL / Raylib / SFML backends).

1. Scope & Threat Model

Before identifying risks, it's essential to define what Shimera is and isn't, in order to frame a relevant analysis rather than a generic one.

Shimera is:

- An **Open Source software library only** (not a service, not an application)
- A library that **handles no user data** (no accounts, no sessions, no personal content)
- A library that **stores no sensitive data** (no database, no persisted user config files)
- A library with **no network service or authentication** (no server, no remote API, no network RCE possible)

Direct consequence for the scope of this map: classic risk categories (auth, session management, personal data leaks, network exposure) are **out of scope**, because they are structurally absent from the library. Shimera's real risk lies elsewhere:

- **Memory robustness:** Shimera directly manipulates GPU resources (OpenGL) via low-level code, so classic C++ security risks (buffer overflow, use-after-free, double-free) apply.
- **Public API boundary:** Shimera is integrated into third-party applications. Unvalidated input can propagate from the client app all the way to the graphics driver.
- **Supply chain / CI-CD integrity:** as an Open Source project with a self-hosted runner, the build infrastructure is its own attack surface, independent of the library's code itself.

This map therefore documents **Shimera's actual protection strategy**: what is protected, why, and how, within these boundaries.

2. Sensitive Modules

These modules are considered sensitive because they directly manipulate native resources (GPU, memory, files) from input provided by the calling code.

Module	Role	Why it's sensitive
Framebuffers (<code>OpenGLFramebuffer</code> and equivalents)	Allocates GPU FBO/RBO from caller-provided dimensions	Sensitive because it directly manipulates GPU resources
Textures (<code>OpenGLTexture</code> and equivalents)	Allocates GPU textures	Same logic as Framebuffer — low-level GPU manipulation remains an area to monitor
Shaders (<code>OpenGLShader</code> and equivalents)	Loads, compiles, and links GLSL programs from files	Takes caller-provided file paths; manages the lifecycle of a GPU resource (<code>glProgram</code>)
EffectPipeline	Orchestrates multiple post-processing effects using double-buffered (ping-pong) FBOs	The module with the most complex internal state in the project; any mismatch between resize and rendering can trigger a crash

Aperçu du début de la security Map

Objectif D: Mesurer, tester et optimiser la performance technique

L'objectif

L'objectif est de définir des mesures clés, d'intégrer différents tests (performances, stress, résilience) au cycle de développement du projet et de les comparer dans des benchmarks pour justifier et démontrer les performances du projet.

Pertinence pour la track technique.

La track vise explicitement à "optimiser la performance à tous les niveaux" et rejette le réflexe "ça tourne vite sur nos machines". Cet objectif impose une ingénierie rigoureuse, pilotée par la mesure et reproductible. Mesurer d'abord, optimiser en fonction des données puis documenter les compromis.

Pertinence pour Shimera.

La performance est capitale pour un projet graphique comme le nôtre. Comme nous manipulons des données de bas niveau, nous devons optimiser chaque étape, que ce soit sur la carte graphique (GPU et shaders) ou sur le processeur (CPU et transferts). Si notre bibliothèque manque de rapidité, elle ne sera jamais adoptée par les développeurs. Ce travail d'optimisation est donc une exigence technique indispensable pour garantir l'utilisabilité de l'outil et nous permet également d'utiliser ces résultats concrets pour attirer et convaincre de nouveaux utilisateurs.

Preuves concrètes

Pour ce faire nous avons mis en place plusieurs types de tests (mémoire, performances, résilience) qui sont chacun documentés sur leurs implémentations. Vous pouvez retrouver la documentation via ce lien :

https://shimerateam.github.io/ShimeraDocs/devdoc/testing/testing_guide

Testing Guide

Introduction

This document describes the different test categories used in Shimera, the associated tools, and the target platforms for each.

1. Memory Tests

These tests detect memory leaks, invalid accesses, and undefined behavior.

Linux - AddressSanitizer (ASan) / LeakSanitizer (LSan)

Compiled with ASan/LSan flags, these tests run on Linux and on WSL.

Windows - CRT Debug Heap

On native Windows, the CRT Debug Heap detects leaks at program exit. No additional flags are required in MSVC Debug builds.

Example

MemoryTest.cpp main.cpp

```
#include "MemoryTest.hpp"
#include "backend/BackendFactory.hpp"

#ifdef _WIN32
```

cpp

Début de la documentation du guide de test

Ces tests peuvent être lancés en local via deux fichiers “run-test.sh” pour Linux et “run-tests.bat” pour Windows. Ils sont également lancés via une GitHub Action sur les branches dev et master pour s’assurer du fonctionnement de la librairie et de ses performances. La GitHub Action lance les tests sur un serveur dédié Quadro RTX 5000.

```
1  ▶ Run ./run-tests.sh
6  Configure and Build -> OK
7  Build Shimer OpenGL -> OK
8  Build Shimer Raylib -> OK
9  Build Shimer SFML -> OK
10 Build OpenGL Tests -> OK
11 Build Raylib Tests -> OK
12 Build SFML Tests -> OK
13 Build OpenGL Resilience Tests -> OK
14 Build Raylib Resilience Tests -> OK
15 Build SFML Resilience Tests -> OK
16 Run OpenGL Tests -> OK
17 Run Raylib Tests -> FAILED
18 INFO: Initializing raylib 6.0
19 INFO: Platform backend: DESKTOP (GLFW)
20 INFO: Supported raylib modules:
21 INFO:   > rcore:..... loaded (mandatory)
22 INFO:   > rlgl:..... loaded (mandatory)
23 INFO:   > rshapes:... loaded (optional)
24 INFO:   > rtextures:. loaded (optional)
25 INFO:   > rtext:..... loaded (optional)
26 INFO:   > rmodels:... loaded (optional)
27 INFO:   > raudio:... loaded (optional)
28 INFO: DISPLAY: Device initialized successfully
29 INFO:   > Display size: 1024 x 768
30 INFO:   > Screen size: 960 x 540
31 INFO:   > Render size: 960 x 540
32 INFO:   > Viewport offsets: 0, 0
33 INFO: GLAD: OpenGL extensions loaded successfully
34 INFO: GL: Supported extensions count: 404
```

Ci GitHub Action “Run Tests”

```

=====Resilience Tests=====
[=====] Running 10 tests from 4 test suites.
[-----] Global test environment set-up.
[-----] 1 test from BackendFactory
[ RUN      ] BackendFactory.CreateBackendWithoutContext
[          OK ] BackendFactory.CreateBackendWithoutContext (0 ms)
[-----] 1 test from BackendFactory (0 ms total)

[-----] 7 tests from ContextTest
[ RUN      ] ContextTest.CreateBackend
[          OK ] ContextTest.CreateBackend (0 ms)
[ RUN      ] ContextTest.CreateFrameBuffer
[          OK ] ContextTest.CreateFrameBuffer (1 ms)
[ RUN      ] ContextTest.CreateFrameBufferWithBadSize
[          OK ] ContextTest.CreateFrameBufferWithBadSize (0 ms)
[ RUN      ] ContextTest.CreateEffectPipeline
[          OK ] ContextTest.CreateEffectPipeline (0 ms)
[ RUN      ] ContextTest.AddEffect
[          OK ] ContextTest.AddEffect (1 ms)
[ RUN      ] ContextTest.GetEffect
[          OK ] ContextTest.GetEffect (0 ms)
[ RUN      ] ContextTest.GetEffectNotFound
[          OK ] ContextTest.GetEffectNotFound (0 ms)
[-----] 7 tests from ContextTest (2 ms total)

[-----] 1 test from IFrameBuffer
[ RUN      ] IFrameBuffer.CreateFrameBufferWithoutContext
[          OK ] IFrameBuffer.CreateFrameBufferWithoutContext (0 ms)
[-----] 1 test from IFrameBuffer (0 ms total)

[-----] 1 test from EffectPipeline
[ RUN      ] EffectPipeline.CreateEffectPipelineWithoutContext

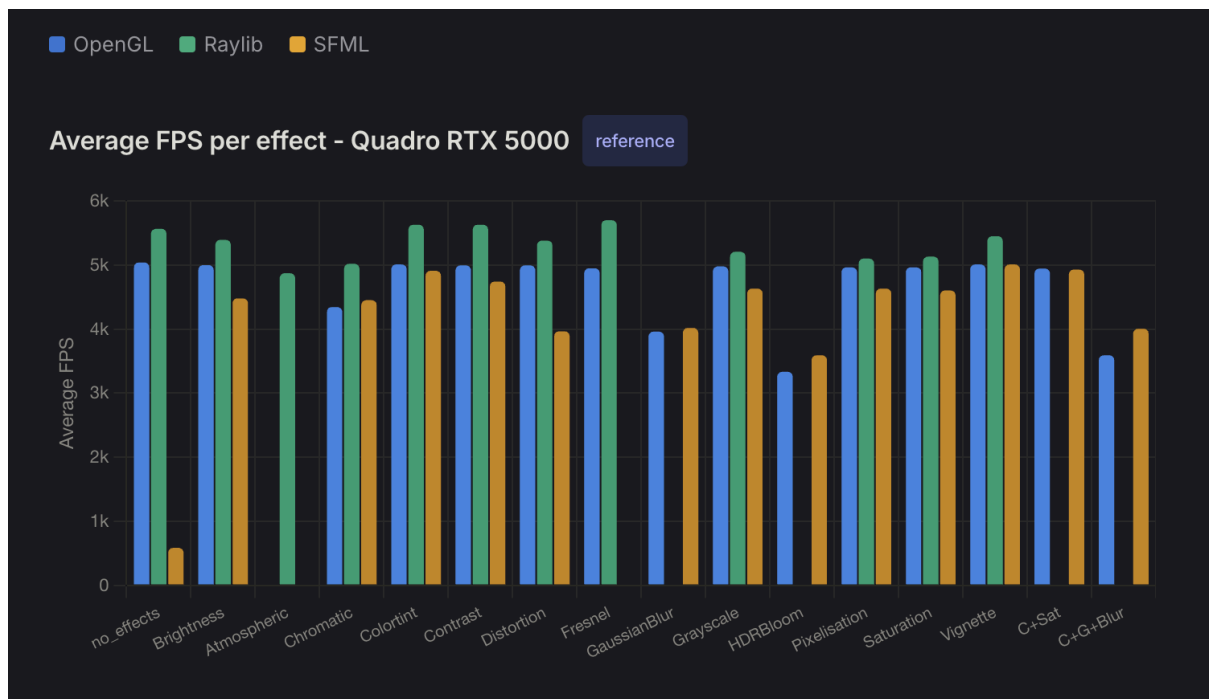
```

Résilience Tests lancés par GitHub Actions

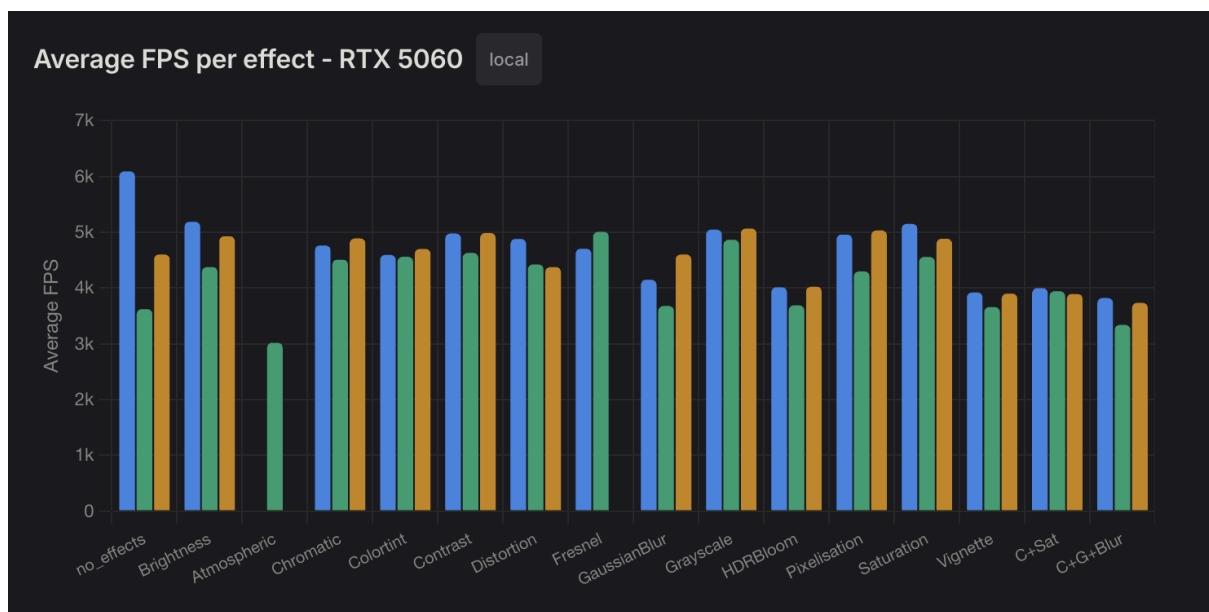
Parmi les tests lancés, il y a également des tests de performance qui sont exportés dans un fichier JSON "benchmark-results.json" qui enregistre toutes les mesures de performance de Shimera. Ces performances sont ensuite mises en avant dans notre documentation sous forme de graphiques pour les partager avec les utilisateurs. Vous pouvez les retrouver via ce lien : <https://shimerateam.github.io/ShimeraDocs/userdoc/performance/benchmark>

```
"Quadro RTX 5000/PCIe/SSE2": {
  "OpenGL": {
    "BrightnessEffect": {
      "avgFps": 5000.0,
      "frames": 5000.0,
      "totalMs": 1000.0,
      "vramUsed": 13568
    },
    "ChromaticAberrationEffect": {
      "avgFps": 4344.0,
      "frames": 5000.0,
      "totalMs": 1151.0,
      "vramUsed": 13568
    },
    "ColortintEffect": {
      "avgFps": 5010.0,
      "frames": 5000.0,
      "totalMs": 998.0,
      "vramUsed": 13568
    },
    "ContrastEffect": {
      "avgFps": 4995.0,
      "frames": 5000.0,
      "totalMs": 1001.0,
      "vramUsed": 13568
    },
    "ContrastEffect, GrayscaleEffect, GaussianBlurEffect": {
      "avgFps": 3591.0,
      "frames": 5000.0,
      "totalMs": 1392.0,
      "vramUsed": 16192
    },
  },
}
```

“benchmark-results.json”



Graphique des performances de la Quadro RTX 5000 dans la documentation des benchmarks



Graphique des performances de la GeForce RTX 5060 dans la documentation des benchmarks



Graphiques de comparaison entre les deux cartes graphiques dans la doc des benchmarks.